
The Role of Weather in French Day-Ahead Electricity Price Forecasting: A Random Forest Approach

— Code Appendix —

Master's Thesis — MSc Data Analysis & AI
EDHEC Business School

**Leo Camberleng
Lyam Oumedjeber**

May 2026

Contents

Overview	2
1 Configuration	3
1.1 src/config.py	3
2 Data Collection	5
2.1 scripts/fetch_entsoe.py	5
2.2 scripts/fetch_era5.py	9
2.3 scripts/fetch_fuels.py	12
3 Feature Engineering	18
3.1 src/preprocessing.py	18
3.2 src/features.py	20
3.3 scripts/build_features.py	25
4 Modelling and Evaluation	26
4.1 src/models.py	26
4.2 src/evaluate.py	28
4.3 src/plots.py	31
5 Trading Backtest	33
5.1 scripts/backtest.py	33
6 2022 Energy Crisis Validation	49
6.1 scripts/test_2022.py	49
7 End-to-End Pipeline	55
7.1 scripts/run_full_pipeline.py	55

Overview

This document contains the complete Python codebase used to produce the results of the thesis “*The Role of Weather in French Day-Ahead Electricity Price Forecasting: A Random Forest Approach*”.

The codebase is organised as follows:

- **Configuration** (`src/config.py`): global paths, API keys, and model hyperparameters.
- **Data Collection** (`scripts/fetch_*.py`): retrieval from ENTSO-E API, ERA5 CDS API, and fuel price sources.
- **Feature Engineering** (`src/features.py`, `src/preprocessing.py`): construction of the 35-feature matrix.
- **Modelling & Evaluation** (`src/models.py`, `src/evaluate.py`): Random Forest and XGBoost training, metrics, Diebold-Mariano tests.
- **Trading Backtest** (`scripts/backtest.py`): day-ahead directional strategy, P&L, Sharpe, drawdown, transaction cost scenarios.
- **2022 Crisis Validation** (`scripts/test_2022.py`): regime-separated evaluation on the 2022 energy crisis.
- **Pipeline** (`scripts/run_full_pipeline.py`): end-to-end orchestration script.

All data sources (ENTSO-E Transparency Platform, ERA5 reanalysis, fuel prices) are publicly available. The feature matrix is reproducible by running `scripts/run_full_pipeline.py` after configuring the API keys.

Chapter 1

Configuration

1.1 src/config.py

Global paths, API credentials (loaded from environment variables), market parameters, and model hyperparameters. Random Forest uses 500 trees with `max_depth=10` and `min_samples_leaf=5`. XGBoost uses 500 estimators with learning rate $\eta = 0.05$ and `max_depth=6`.

```
1 from pathlib import Path
  from dotenv import load_dotenv
  import os

  load_dotenv()

6
  ROOT = Path(__file__).resolve().parents[1]

  # Data paths
  RAW_DIR = ROOT / "data" / "raw"
11 INTERIM_DIR = ROOT / "data" / "interim"
  PROCESSED_DIR = ROOT / "data" / "processed"

  # Output paths
  FIGURES_DIR = ROOT / "outputs" / "figures"
16 TABLES_DIR = ROOT / "outputs" / "tables"
  METRICS_DIR = ROOT / "outputs" / "metrics"

  # API keys
  ENTSOE_API_KEY = os.getenv("ENTSOE_API_KEY", "")
21 CDS_API_KEY = os.getenv("CDS_API_KEY", "")
  CDS_API_URL = os.getenv("CDS_API_URL", "https://cds.climate.
    copernicus.eu/api")

  # Market config
  BIDDING_ZONE = "10YFR-RTE-----C" # France RTE
26 TIMEZONE = "Europe/Paris"

  # Modelling
```

```
TARGET_COL = "price_da_eur_mwh"
TEST_MONTHS = 12
31 RANDOM_SEED = 42

RF_PARAMS = {
    "n_estimators": 500,
    "max_depth": 10,
    36 "min_samples_leaf": 5,
    "n_jobs": -1,
    "random_state": RANDOM_SEED,
}

41 XGB_PARAMS = {
    "n_estimators": 500,
    "max_depth": 6,
    "learning_rate": 0.05,
    "subsample": 0.8,
    46 "colsample_bytree": 0.8,
    "random_state": RANDOM_SEED,
}
```

Listing 1.1: `src/config.py` — Project configuration and hyperparameters

Chapter 2

Data Collection

2.1 scripts/fetch_entsoe.py

Retrieves hourly day-ahead prices, load forecasts, actual generation by source, and cross-border physical flows for France (bidding zone 10YFR-RTE---C) from the ENTSO-E Transparency Platform using the `entsoe-py` Python library.

```
"""
2 Fetch French day-ahead prices, load forecast, generation mix,
  cross-border flows, and nuclear availability from ENTSO-E.
  Saves raw data to data/raw/ as parquet files.
"""

import ssl, urllib3
7 ssl._create_default_https_context = ssl._create_unverified_context
  urllib3.disable_warnings(urllib3.exceptions.InsecureRequestWarning)

import pandas as pd
from entsoe import EntsoePandasClient
12 from src.config import ENTSOE_API_KEY, BIDDING_ZONE, RAW_DIR

RAW_DIR.mkdir(parents=True, exist_ok=True)

START = pd.Timestamp("2018-01-01", tz="Europe/Paris")
17 END   = pd.Timestamp("2025-04-30", tz="Europe/Paris")

client = EntsoePandasClient(api_key=ENTSOE_API_KEY)

#
-----

22 # Neighbours for cross-border flows (ENTSO-E bidding zone codes)
#
-----

NEIGHBOURS = {
    "DE": "10Y1001A1001A83F",    # Germany (DE-LU)
```

```

27     "ES": "10YES-REE-----0",    # Spain
    "IT": "10YIT-GRTN-----W",    # Italy        use two-letter code if
        zone fails
    "GB": "10YGB-----A",    # Great Britain
    "CH": "10YCH-SWISSGRID--D",    # Switzerland    use two-letter
        code if zone fails
    "BE": "10YBE-----2",    # Belgium
}

32
# Fallback two-letter country codes for borders that fail with zone
  codes
NEIGHBOURS_CC = {
    "IT": "IT",
    "CH": "CH",
37 }

def fetch_and_save(name, fn, *args, **kwargs):
    print(f"Fetching {name}...")
42     try:
        data = fn(*args, **kwargs)
        path = RAW_DIR / f"{name}.parquet"
        if isinstance(data, pd.Series):
            data.to_frame().to_parquet(path)
47         else:
            data.to_parquet(path)
            print(f"    Saved {len(data)} rows -> {path}")
            return data
    except Exception as e:
52         print(f"    ERROR fetching {name}: {e}")
        return None

#
-----

57 # Core ENTSO-E data (unchanged from Phase 1, extended to
    2025-04-30)
#
-----

fetch_and_save(
    "prices_da",
    client.query_day_ahead_prices,
62     BIDDING_ZONE, start=START, end=END,
)

fetch_and_save(

```

```

        "load_forecast",
67     client.query_load_forecast,
        BIDDING_ZONE, start=START, end=END,
    )

    fetch_and_save(
72     "generation_forecast",
        client.query_generation_forecast,
        BIDDING_ZONE, start=START, end=END,
    )

77     fetch_and_save(
        "generation_actual",
        client.query_generation,
        BIDDING_ZONE, start=START, end=END,
    )
82     #
    -----

    # Cross-border physical flows (net exports FR      neighbour)
    #
    -----

    print("\nFetching cross-border flows...")
87     flow_frames = {}
    for country, zone in NEIGHBOURS.items():
        # Try zone code first, then two-letter CC fallback
        codes_to_try = [zone]
        if country in NEIGHBOURS_CC:
92             codes_to_try.append(NEIGHBOURS_CC[country])
        success = False
        for code in codes_to_try:
            try:
                exports = client.query_crossborder_flows(BIDDING_ZONE,
                    code, start=START, end=END)
97                 imports = client.query_crossborder_flows(code,
                    BIDDING_ZONE, start=START, end=END)
                net = exports - imports
                net.name = f"flow_net_fr_{country.lower()}_mw"
                flow_frames[country] = net
                print(f"  FR-{country}: {len(net)} rows (code: {code})"
                    )
            except Exception as e:
102                 success = True
                break
            except Exception as e:
                pass
        if not success:

```



```

107         print(f" WARNING FR-{country}: could not fetch with any
              code")

if flow_frames:
    df_flows = pd.concat(flow_frames.values(), axis=1)
    df_flows = df_flows.resample("h").mean()
112    df_flows.to_parquet(RAW_DIR / "crossborder_flows.parquet")
    print(f" Saved crossborder_flows.parquet ({df_flows.shape})")

#
-----

# Nuclear unavailability / availability proxy
117 # Using installed generation capacity (aggregated) to get declared
    capacity,
# then computing availability ratio vs actual generation.
#
-----

print("\nFetching nuclear installed capacity (aggregated)...")
try:
122     nuclear_capacity = client.
        query_installed_generation_capacity_aggregated(
            BIDDING_ZONE, start=START, end=END, psr_type="B14" # B14 =
                        Nuclear
        )
# Result is typically annual/semi-annual forward-fill to
    hourly
if isinstance(nuclear_capacity, pd.DataFrame):
127     nuclear_capacity = nuclear_capacity.iloc[:, 0]
    nuclear_capacity.name = "nuclear_installed_capacity_mw"
    nuclear_capacity = nuclear_capacity.resample("h").ffill()
    nuclear_capacity.to_frame().to_parquet(RAW_DIR / "
        nuclear_capacity.parquet")
    print(f" Saved nuclear_capacity.parquet ({len(nuclear_capacity
        )} rows)")
132 except Exception as e:
    print(f" WARNING nuclear capacity: {e}")
    print(" Will use fixed 63,000 MW as fallback in features.py")

print("\nENTSO-E download complete.")

```

Listing 2.1: scripts/fetch_entsoe.py — ENTSO-E data retrieval

2.2 scripts/fetch_era5.py

Downloads ERA5 reanalysis weather data (2m temperature, 10m wind speed, solar irradiance, precipitation) for metropolitan France from the Copernicus Climate Data Store using the cdsapi library.

```

"""
Download ERA5 hourly weather data for France via the Copernicus CDS
API.
Requests are split month-by-month to stay within the CDS size limit
.
4 Saves to data/raw/era5_YYYY_MM.nc, then merges to data/raw/
era5_france.parquet.
"""
import sys
sys.path.insert(0, ".")

9 import ssl, urllib3
ssl._create_default_https_context = ssl._create_unverified_context
urllib3.disable_warnings(urllib3.exceptions.InsecureRequestWarning)

import cdsapi
14 import xarray as xr
import pandas as pd
from pathlib import Path
from src.config import CDS_API_KEY, CDS_API_URL, RAW_DIR

19 RAW_DIR.mkdir(parents=True, exist_ok=True)

AREA = [51.5, -5.5, 41.5, 10.0] # [N, W, S, E]      France bounding
box

VARIABLES = [
24     "2m_temperature",
    "10m_u_component_of_wind",
    "10m_v_component_of_wind",
    "surface_solar_radiation_downwards",
    "total_precipitation",
29 ]

YEARS = range(2018, 2026) # 2018-2025 inclusive (covers up to
2025-12)
MONTHS = range(1, 13)

34 client = cdsapi.Client(url=CDS_API_URL, key=CDS_API_KEY, quiet=True
)

def nc_to_dataframe(nc_path: Path) -> pd.DataFrame:

```

```

import zipfile, io
# CDS API v2 returns zip archives containing instant + accum nc
  files
if zipfile.is_zipfile(nc_path):
    frames = []
    with zipfile.ZipFile(nc_path) as zf:
        for name in zf.namelist():
            if not name.endswith(".nc"):
                continue
            data = zf.read(name)
            ds = xr.open_dataset(io.BytesIO(data))
            frames.append(ds.mean(dim=["latitude", "longitude"]
                                ).to_dataframe())
df = pd.concat(frames, axis=1)
df = df.loc[:, ~df.columns.duplicated()]
else:
    ds = xr.open_dataset(nc_path)
    df = ds.mean(dim=["latitude", "longitude"]).to_dataframe()

df.index = pd.to_datetime(df.index).tz_localize("UTC").
    tz_convert("Europe/Paris")
if "u10" in df.columns and "v10" in df.columns:
    df["wind_speed_10m"] = (df["u10"] ** 2 + df["v10"] ** 2) **
        0.5
    df = df.drop(columns=["u10", "v10"])
df = df.rename(columns={
    "t2m": "temperature_2m",
    "ssrd": "solar_radiation",
    "tp": "precipitation",
})
if "temperature_2m" in df.columns:
    df["temperature_2m"] = df["temperature_2m"] - 273.15
return df

frames = []
total = len(YEARS) * len(list(MONTHS))
done = 0

for year in YEARS:
    for month in MONTHS:
        nc_path = RAW_DIR / f"era5_{year}_{month:02d}.nc"
        done += 1
        label = f"{year}-{month:02d} [{done}/{total}]"

        if nc_path.exists():
            print(f" {label} already downloaded, loading...")
        else:

```

```

import calendar
_, n_days = calendar.monthrange(year, month)
84 print(f"  Downloading ERA5 {label}...", flush=True)
try:
    client.retrieve(
        "reanalysis-era5-single-levels",
        {
89         "product_type": "reanalysis",
        "variable": VARIABLES,
        "year": str(year),
        "month": f"{month:02d}",
        "day": [f"{d:02d}" for d in range(1, n_days
94             + 1)],
        "time": [f"{h:02d}:00" for h in range(24)],
        "area": AREA,
        "format": "netcdf",
        },
        str(nc_path),
99    )
except Exception as e:
    print(f"  ERROR {label}: {e}")
    continue

104 try:
    frames.append(nc_to_dataframe(nc_path))
except Exception as e:
    print(f"  Parse error {label}: {e}")

109 if frames:
    df_all = pd.concat(frames).sort_index()
    out_parquet = RAW_DIR / "era5_france.parquet"
    df_all.to_parquet(out_parquet)
    print(f"ERA5 merged -> {out_parquet} ({len(df_all):,} rows)")
114 else:
    print("No data downloaded.")

```

Listing 2.2: scripts/fetch_era5.py — ERA5 weather data download

2.3 scripts/fetch_fuels.py

Retrieves TTF natural gas and ARA coal price series from public market data sources and aligns them to the hourly electricity price index.

```

"""
Fetch fuel price data (TTF natural gas, EUA CO2, ARA coal) from
free sources.
Saves to data/raw/fuel_prices.parquet with daily granularity,
forward-filled to hourly in build_interim.py.
5
Sources (all free / open):
    TTF gas   : Yahoo Finance (ticker TTF=F         front-month futures,
        EUR/MWh)
    EUA CO2   : Ember Climate (open CSV         monthly update)
    Coal ARA  : Yahoo Finance (ticker MTF=F         ARA coal front-month
        , USD/t         EUR/t)
10
Fallback strategy:
    If yfinance fails for any ticker, the column is skipped with a
        warning.
    Clean Spreads in features.py are computed only when all three
        columns exist.
"""
15
import ssl
import urllib3
import requests
import pandas as pd
import numpy as np
20
from src.config import RAW_DIR

#
-----

# SSL workaround      corporate/Windows environments often block
    cert verification
#
-----

25
ssl._create_default_https_context = ssl._create_unverified_context
urllib3.disable_warnings(urllib3.exceptions.InsecureRequestWarning)

_SESSION = requests.Session()
_SESSION.verify = False
30

RAW_DIR.mkdir(parents=True, exist_ok=True)

START = "2018-01-01"
END    = "2025-04-30"

```

```

35
#
-----

# Helper
#
-----

40 def _yf_daily(ticker: str, start: str, end: str, col_name: str) ->
    pd.Series | None:
    """Download a daily close series from Yahoo Finance via
        yfinance (curl_cffi backend)."""
    try:
        import yfinance as yf
        raw = yf.download(
45             ticker, start=start, end=end,
            auto_adjust=True, progress=False,
        )
        if raw.empty:
            print(f" WARNING: yfinance returned empty data for {
                ticker}")
            return None
50         s = raw["Close"].squeeze()
        s.name = col_name
        s.index = pd.to_datetime(s.index).tz_localize("UTC")
        print(f" {ticker} ({col_name}): {len(s)} daily rows")
55         return s
    except Exception as e:
        print(f" WARNING: yfinance failed for {ticker}: {e}")
        return None

60

def _ember_eua(start: str, end: str) -> pd.Series | None:
    """Download EU ETS (EUA) carbon price from Ember Climate open
        data.

    Ember publishes a daily EUA price CSV updated monthly.
    URL: https://ember-climate.org/data/data-tools/carbon-price-
65         viewer/
    Fallback direct link to the underlying CSV.
    """
    urls = [
        # Ember direct CSV
70         "https://ember-climate.org/app/uploads/2022/01/EU-ETS-
            carbon-prices.csv",
        # Backup: Sandbag / Carbon Monitor aggregation
        "https://raw.githubusercontent.com/openclimatedata/eu-ets-

```

```

        carbon-price/main/data/eu-ets-carbon-price.csv",
    ]
    for url in urls:
        try:
            resp = _SESSION.get(url, timeout=30)
            resp.raise_for_status()
            from io import StringIO
            df = pd.read_csv(StringIO(resp.text))
            # Detect date and price columns flexibly
            date_col = next((c for c in df.columns if "date" in c.
                             lower()), df.columns[0])
            price_col = next(
                (c for c in df.columns if any(x in c.lower() for x
                                                in ["price", "eur", "eua", "carbon"])),
                df.columns[-1],
            )
            s = pd.to_numeric(df[price_col], errors="coerce")
            s.index = pd.to_datetime(df[date_col], errors="coerce")
            s = s.dropna()
            s.index = s.index.tz_localize("UTC")
            s.name = "eua_eur_t"
            s = s.sort_index().loc[start:end]
            if len(s) > 100:
                print(f"  EUA (Ember): {len(s)} daily rows from {
                    url}")
                return s
        except Exception as e:
            print(f"  WARNING Ember URL {url}: {e}")

    # Final fallback: yfinance EUA futures
    print("  Falling back to yfinance for EUA (EMISS.L or CO2.DE)
        ...")
    for ticker in ["CO2.DE", "EMISS.L", "EUA=F"]:
        s = _yf_daily(ticker, start, end, "eua_eur_t")
        if s is not None and len(s) > 100:
            return s

    return None

#
-----

# Alternative: FRED API (St. Louis Fed)      uses requests, no
curl_cffi
#
-----

```

```

FRED_BASE = "https://fred.stlouisfed.org/graph/fredgraph.csv"

def _fred_series(series_id: str, col_name: str, start: str, end:
str) -> pd.Series | None:
    """Fetch a monthly series from FRED (no API key required for
    CSV).
    FRED CSV format: observation_date,<series_id>
    """
    try:
        url = f"{FRED_BASE}?id={series_id}"
        resp = _SESSION.get(url, timeout=30)
        resp.raise_for_status()
        from io import StringIO
        df = pd.read_csv(StringIO(resp.text))
        # FRED uses 'observation_date' as the date column
        date_col = df.columns[0]
        val_col = df.columns[1]
        df[date_col] = pd.to_datetime(df[date_col], errors="coerce"
        )
        df = df.dropna(subset=[date_col])
        df = df.set_index(date_col)
        s = pd.to_numeric(df[val_col], errors="coerce").dropna()
        s.index = s.index.tz_localize("UTC")
        s = s.loc[start:end]
        s.name = col_name
        if len(s) > 10:
            print(f"  FRED {series_id} ({col_name}): {len(s)} rows"
            )
        return s
    except Exception as e:
        print(f"  WARNING FRED {series_id}: {e}")
        return None

#
# -----

# Main fetch
#
# -----

print("Fetching fuel prices...")

frames = []

# TTF gas      try yfinance first, fallback to FRED EU gas price
index

```



```

150 ttf = _yf_daily("TTF=F", START, END, "ttf_eur_mwh")
    if ttf is None:
        # FRED: PNGASEUUSD = EU natural gas import price (USD/MMBtu)
        # convert to EUR/MWh
        # 1 MMBtu      0.2931 MWh      divide by 0.2931 to get USD/MWh
        # multiply by 0.92 EUR/USD
        ttf_usd = _fred_series("PNGASEUUSD", "ttf_eur_mwh", START, END
                                )
155     if ttf_usd is not None:
        ttf = (ttf_usd / 0.2931 * 0.92).resample("D").ffill()
        ttf.name = "ttf_eur_mwh"
        print(f"  TTF (FRED proxy, EUR/MWh): {len(ttf)} rows")
    if ttf is not None:
160         frames.append(ttf)

    # EUA CO2      try Ember then yfinance
    eua = _ember_eua(START, END)
    if eua is None:
165         # FRED doesn't have EUA      use a fixed rolling estimate based
        # on known ranges
        # EUA ranged ~15-30 EUR/t (2018-2020), ~25-50 (2021), ~60-100
        # (2022-2023), ~50-70 (2024)
        # This is a rough proxy      acknowledge limitation in thesis
        print("  INFO: EUA not available from automated sources.
              Skipping clean spreads.")
    if eua is not None:
170         frames.append(eua)

    # ARA coal      try yfinance, fallback to FRED coal price
    coal_usd = _yf_daily("MTF=F", START, END, "coal_eur_t")
    if coal_usd is None:
175         # FRED: PCOALAUUSD = Australian coal price (USD/metric ton)
        # closest proxy to ARA
        coal_usd = _fred_series("PCOALAUUSD", "coal_eur_t", START, END
                                )
        if coal_usd is not None:
            coal_usd = (coal_usd * 0.92).resample("D").ffill()
            coal_usd.name = "coal_eur_t"
180             print(f"  Coal (FRED proxy, EUR/t): {len(coal_usd)} rows")
    if coal_usd is not None:
        frames.append(coal_usd)

    #
    -----

185 # Merge & save
    #
    -----

```

```

if frames:
    df_fuels = pd.concat(frames, axis=1).sort_index()
    # Keep only the requested date range
190 df_fuels = df_fuels.loc[START:END]
    # Ensure daily index (some tickers have duplicate timestamps)
    df_fuels = df_fuels[~df_fuels.index.duplicated(keep="last")]
    df_fuels = df_fuels.resample("D").last()

195 out = RAW_DIR / "fuel_prices.parquet"
    df_fuels.to_parquet(out)
    print(f"\nSaved fuel_prices.parquet -> {out}")
    print(f"  Shape: {df_fuels.shape}")
    print(f"  Columns: {list(df_fuels.columns)}")
200 print(f"  Period: {df_fuels.index.min()} -> {df_fuels.index.max()}"
        f"}")
    print(df_fuels.describe().round(2))
else:
    print("\nWARNING: No fuel price data could be fetched.")
    print("  Clean Spreads will be skipped in the feature matrix.")
205 print("  Models will still train without fuel spread features."
        )

```

Listing 2.3: scripts/fetch_fuels.py — Fuel price data retrieval

Chapter 3

Feature Engineering

3.1 src/preprocessing.py

Data cleaning, timestamp alignment, and interpolation of missing values. All series are aligned to UTC+1 (Europe/Paris) and reindexed to a complete hourly grid.

```
import pandas as pd
from entsoe import EntsoePandasClient
from src.config import ENTSOE_API_KEY, BIDDING_ZONE, TIMEZONE,
    RAW_DIR

5
def fetch_day_ahead_prices(start: pd.Timestamp, end: pd.Timestamp)
-> pd.Series:
    client = EntsoePandasClient(api_key=ENTSOE_API_KEY)
    prices = client.query_day_ahead_prices(BIDDING_ZONE, start=
        start, end=end)
    prices.name = "price_da_eur_mwh"
10    return prices

def fetch_load_forecast(start: pd.Timestamp, end: pd.Timestamp) ->
pd.DataFrame:
    client = EntsoePandasClient(api_key=ENTSOE_API_KEY)
15    return client.query_load_forecast(BIDDING_ZONE, start=start,
        end=end)

def fetch_generation_forecast(start: pd.Timestamp, end: pd.
Timestamp) -> pd.DataFrame:
    client = EntsoePandasClient(api_key=ENTSOE_API_KEY)
20    return client.query_generation_forecast(BIDDING_ZONE, start=
        start, end=end)

def load_era5_weather(path=None) -> pd.DataFrame:
```

```
    """Load ERA5 weather data from a parquet file downloaded via
        cdsapi."""
25  if path is None:
        path = RAW_DIR / "era5_france.parquet"
    return pd.read_parquet(path)

30  def clean_prices(prices: pd.Series) -> pd.Series:
        prices = prices.resample("h").mean()
        prices = prices.clip(lower=-500, upper=3000)
        prices = prices.interpolate(method="time", limit=3)
    return prices

35

    def merge_features(prices: pd.Series, load: pd.DataFrame, weather:
        pd.DataFrame) -> pd.DataFrame:
        df = prices.to_frame()
        df = df.join(load.resample("h").mean(), how="left")
40  df = df.join(weather.resample("h").mean(), how="left")
        df.index = df.index.tz_convert(TIMEZONE)
    return df.sort_index()
```

Listing 3.1: src/preprocessing.py — Data cleaning and alignment

3.2 src/features.py

Constructs the full 35-feature matrix from the cleaned data sources. Key features include: calendar encodings (sine/cosine), price lags (24h, 48h, 168h), rolling means, nuclear availability ratio, Heating Degree Days (HDD, threshold 17°C), Cooling Degree Days (CDD, threshold 22°C), wind power proxy (v^3), and the Weather Stress Index ($WSI = HDD / \text{wind speed}$).

```

import numpy as np
import pandas as pd

3
# French nuclear installed capacity (MW)      source: RTE Bilan
#      lectrique      2023
FR_NUCLEAR_CAPACITY_MW = 63_000

# Clean spread parameters      CCGT gas, coal plant, source: RTE/
#      ENTSO-E

8 HEAT_RATE_GAS = 2.0      # MWh_gas per MWh_elec (CCGT)
HEAT_RATE_COAL = 2.5      # MWh_coal per MWh_elec
EMISSION_FACTOR_GAS = 0.35 # tCO2 per MWh_elec (CCGT)
EMISSION_FACTOR_COAL = 0.95 # tCO2 per MWh_elec (coal)

13 # HDD threshold      17 C is the standard French heating threshold
#      (source: RTE)
HDD_THRESHOLD = 17.0
CDD_THRESHOLD = 22.0

18 def add_calendar_features(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy()
    df["hour"] = df.index.hour
    df["dayofweek"] = df.index.dayofweek
    df["month"] = df.index.month
23    df["is_weekend"] = (df.index.dayofweek >= 5).astype(int)
    df["hour_sin"] = np.sin(2 * np.pi * df["hour"] / 24)
    df["hour_cos"] = np.cos(2 * np.pi * df["hour"] / 24)
    df["month_sin"] = np.sin(2 * np.pi * df["month"] / 12)
    df["month_cos"] = np.cos(2 * np.pi * df["month"] / 12)
28    return df

def add_price_lags(df: pd.DataFrame, target: str = "
price_da_eur_mwh") -> pd.DataFrame:
    df = df.copy()
33    for lag in [24, 48, 168]:
        df[f"{target}_lag{lag}h"] = df[target].shift(lag)
        df[f"{target}_roll24h_mean"] = df[target].shift(24).rolling(24)
            .mean()
        df[f"{target}_roll168h_mean"] = df[target].shift(24).rolling

```

```

    (168).mean()
    return df
38

def add_weather_features(df: pd.DataFrame) -> pd.DataFrame:
    """Compute derived weather features from ERA5 columns.

43    HDD threshold = 17 C      standard French heating threshold (
        RTE).
    This is higher than the German threshold (~15 C ) reflecting
        the stronger
    thermosensitivity of French electric heating (~2.4 GW/ C in
        winter).
    """
    df = df.copy()
48    if "temperature_2m" in df.columns:
        df["hdd"] = np.maximum(0, HDD_THRESHOLD - df["
            temperature_2m"])
        df["cdd"] = np.maximum(0, df["temperature_2m"] -
            CDD_THRESHOLD)
    if "wind_speed_10m" in df.columns:
        df["wind_power_proxy"] = df["wind_speed_10m"] ** 3
53    return df

def add_weather_stress_index(df: pd.DataFrame) -> pd.DataFrame:
    """Composite index combining cold spells and low wind      high
        values signal
58    peak demand with low renewable generation, a key driver of
        French price spikes."""
    df = df.copy()
    cold = df.get("hdd", pd.Series(0, index=df.index))
    wind = df.get("wind_speed_10m", pd.Series(1, index=df.index)).
        clip(lower=0.1)
    df["weather_stress_index"] = cold / wind
63    return df

def add_nuclear_availability(df: pd.DataFrame) -> pd.DataFrame:
    """Nuclear availability ratio = actual generation / installed
        capacity.

68    Proxy for the fraction of the French nuclear fleet online.
    France has ~63 GW installed (RTE 2023). A ratio below 0.7
        signals
    significant outages and is associated with higher prices.
    """
73    df = df.copy()

```

```

    if "gen_nuclear_mw" in df.columns:
        df["nuclear_avail_ratio"] = (
            df["gen_nuclear_mw"] / FR_NUCLEAR_CAPACITY_MW
        ).clip(0, 1)
78 return df

def add_fuel_spreads(df: pd.DataFrame, target: str = "
price_da_eur_mwh") -> pd.DataFrame:
    """Compute Clean Spark Spread and Clean Dark Spread when fuel
    price columns exist.
83

    CSS = Price_power      (TTF_gas      heat_rate_gas)      (EUA
        emission_factor_gas)
    CDS = Price_power      (Coal      heat_rate_coal)      (EUA
        emission_factor_coal)

    Columns expected (from fetch_fuels.py):
88      ttf_eur_mwh      TTF natural gas day-ahead price in EUR/MWh
      eua_eur_t      EU ETS carbon price in EUR/tCO2
      coal_eur_t      ARA coal price in EUR/tonne
    """

    df = df.copy()
93 price = df.get(target)
    ttf = df.get("ttf_eur_mwh")
    eua = df.get("eua_eur_t")
    coal = df.get("coal_eur_t")

98 if price is not None and ttf is not None and eua is not None:
    df["clean_spark_spread"] = (
        price - (ttf * HEAT_RATE_GAS) - (eua *
            EMISSION_FACTOR_GAS)
    )

103 if price is not None and coal is not None and eua is not None:
    # coal_eur_t      coal_eur_mwh via heat_rate (1 tonne
        6.978 MWh_th)
    coal_eur_mwh = coal / 6.978
    df["clean_dark_spread"] = (
        price - (coal_eur_mwh * HEAT_RATE_COAL) - (eua *
            EMISSION_FACTOR_COAL)
108    )

    return df

113 # Columns produced by weather-related feature functions      used to
    split

```

```

# Mod le B (no weather) from Mod le C (with weather) in the
  pipeline.
WEATHER_FEATURE_COLS = [
    "temperature_2m",
    "wind_speed_10m",
118    "solar_radiation",
    "precipitation",
    "hdd",
    "cdd",
    "wind_power_proxy",
123    "weather_stress_index",
]

def build_feature_matrix(
128    df: pd.DataFrame,
    target: str = "price_da_eur_mwh",
    include_weather: bool = True,
) -> tuple[pd.DataFrame, pd.Series]:
    """Build the full feature matrix.

133    Args:
        df: Merged interim DataFrame (output of build_interim.py).
        target: Name of the price column to predict.
        include_weather: If False, ERA5/weather-derived columns are
            dropped
138        (used to train Mod le B      RF sans m t o).

    Returns:
        (X, y) ready for model training.
    """
143    df = add_calendar_features(df)
    df = add_price_lags(df, target)
    df = add_weather_features(df)
    df = add_weather_stress_index(df)
    df = add_nuclear_availability(df)
148    df = add_fuel_spreads(df, target)

    if not include_weather:
        drop_weather = [c for c in WEATHER_FEATURE_COLS if c in df.
            columns]
        df = df.drop(columns=drop_weather)
153

    # Drop columns that are more than 50% NaN (e.g. offshore wind
        in France)
    thresh = len(df) * 0.5
    df = df.dropna(axis=1, thresh=int(thresh))

```



```
158     # Forward-fill remaining small gaps (hydro, generation, fuel
        prices)
    df = df.ffill(limit=3)

    # Only drop rows where target or core lag features are missing
    key_cols = [target, f"{target}_lag24h", f"{target}_lag168h"]
163    key_cols = [c for c in key_cols if c in df.columns]
    df = df.dropna(subset=key_cols)

    X = df.drop(columns=[target])
    y = df[target]
168    return X, y
```

Listing 3.2: `src/features.py` — Feature matrix construction (35 features)

3.3 scripts/build_features.py

Orchestration script that calls the preprocessing and feature engineering modules to produce the final `features.parquet` file.

```

"""
2 Build final feature matrix from interim merged data.
  Output: data/processed/features.parquet
"""
import sys
sys.path.insert(0, ".")
7
import pandas as pd
from src.config import INTERIM_DIR, PROCESSED_DIR
from src.features import build_feature_matrix
12 PROCESSED_DIR.mkdir(parents=True, exist_ok=True)

print("Loading interim data...")
df = pd.read_parquet(INTERIM_DIR / "merged.parquet")
print(f"   {len(df):,} rows, {df.shape[1]} columns")
17
print("Engineering features...")
X, y = build_feature_matrix(df)
print(f"   Feature matrix: {X.shape[0]:,} rows x {X.shape[1]}
      features")
print(f"   Features: {list(X.columns)}")
22
out = PROCESSED_DIR / "features.parquet"
df_out = X.copy()
df_out["price_da_eur_mwh"] = y
df_out.to_parquet(out)
27 print(f"Saved -> {out}")

```

Listing 3.3: `scripts/build_features.py` — Feature pipeline orchestration

Chapter 4

Modelling and Evaluation

4.1 src/models.py

Wraps scikit-learn's `RandomForestRegressor` and XGBoost's `XGBRegressor` with a common interface. Handles the strict temporal train/test split (no shuffling).

```
import pandas as pd
from sklearn.ensemble import RandomForestRegressor
3 from xgboost import XGBRegressor
from src.config import RF_PARAMS, XGB_PARAMS

def train_random_forest(X_train: pd.DataFrame, y_train: pd.Series)
-> RandomForestRegressor:
8     model = RandomForestRegressor(**RF_PARAMS)
    model.fit(X_train, y_train)
    return model

13 def train_xgboost(X_train: pd.DataFrame, y_train: pd.Series) ->
    XGBRegressor:
    model = XGBRegressor(**XGB_PARAMS)
    model.fit(X_train, y_train, verbose=False)
    return model

18

def naive_forecast(y: pd.Series, lag: int = 168) -> pd.Series:
    """Same-hour previous week benchmark (lag-168h).

    The roadmap defines the naive benchmark as  $\text{price}(h) = \text{price}(h - 168)$ ,
    23 i.e. the same hour from the previous week. This is a stronger
        baseline
        than lag-24h because electricity prices exhibit strong weekly
        seasonality.
    """
```

```
return y.shift(lag)
```

Listing 4.1: `src/models.py` — Model training wrappers

4.2 src/evaluate.py

Computes MAE, RMSE, sMAPE, R^2 , and hit ratio on the test set. Implements the Diebold-Mariano test with Harvey-Leybourne-Newbold correction for pairwise model comparison.

```

import json
import numpy as np
import pandas as pd
4 from pathlib import Path
from scipy import stats
from src.config import METRICS_DIR

9 def mae(y_true: pd.Series, y_pred: pd.Series) -> float:
    return float(np.mean(np.abs(y_true - y_pred)))

def rmse(y_true: pd.Series, y_pred: pd.Series) -> float:
14 return float(np.sqrt(np.mean((y_true - y_pred) ** 2)))

def smape(y_true: pd.Series, y_pred: pd.Series) -> float:
    """Symmetric MAPE      robust to near-zero and negative prices."
    """
19     denom = (np.abs(y_true) + np.abs(y_pred)) / 2
    mask = denom > 0
    return float(np.mean(np.abs(y_true[mask] - y_pred[mask]) /
        denom[mask]) * 100)

24 def r2(y_true: pd.Series, y_pred: pd.Series) -> float:
    ss_res = np.sum((y_true - y_pred) ** 2)
    ss_tot = np.sum((y_true - y_true.mean()) ** 2)
    return float(1 - ss_res / ss_tot)

29 def hit_ratio(y_true: pd.Series, y_pred: pd.Series) -> float:
    """Directional accuracy: % of hours where price movement
        direction is correct."""
    prev = y_true.shift(1).dropna()
    actual_dir = np.sign(y_true.loc[prev.index] - prev)
34    pred_dir = np.sign(y_pred.loc[prev.index] - prev)
    return float((actual_dir == pred_dir).mean() * 100)

def dm_test(
39     y_true: pd.Series,
    y_pred1: pd.Series,
    y_pred2: pd.Series,

```

```

    h: int = 1,
) -> tuple[float, float]:
44     """
    Diebold-Mariano test (1995)      H0: equal predictive accuracy (
        MAE-based).
    Returns (DM statistic, two-sided p-value).
    A negative DM stat means model 1 is MORE accurate than model 2.
    """
49     e1 = y_true - y_pred1
    e2 = y_true - y_pred2
    d = np.abs(e1.values) - np.abs(e2.values)
    n = len(d)
    d_mean = d.mean()

54     # Newey-West long-run variance with h-1 lags
    gamma0 = np.var(d, ddof=1)
    gamma_sum = sum(
        (1 - k / h) * np.cov(d[k:], d[:-k])[0, 1]
59     for k in range(1, h)
    ) if h > 1 else 0
    var_d = (gamma0 + 2 * gamma_sum) / n

    if var_d <= 0:
64         return float("nan"), float("nan")

    dm_stat = d_mean / np.sqrt(var_d)
    # Harvey, Leybourne & Newbold (1997) small-sample correction
    hln_factor = np.sqrt((n + 1 - 2 * h + h * (h - 1) / n) / n)
69     dm_stat_adj = dm_stat * hln_factor
    p_value = float(2 * (1 - stats.norm.cdf(abs(dm_stat_adj))))
    return float(dm_stat_adj), p_value

74 def compute_all(y_true: pd.Series, y_pred: pd.Series) -> dict:
    return {
        "mae": mae(y_true, y_pred),
        "rmse": rmse(y_true, y_pred),
        "smape": smape(y_true, y_pred),
79     "r2": r2(y_true, y_pred),
        "hit_ratio": hit_ratio(y_true, y_pred),
    }

84 def save_metrics(metrics: dict, name: str) -> Path:
    METRICS_DIR.mkdir(parents=True, exist_ok=True)
    path = METRICS_DIR / f"{name}.json"
    with open(path, "w") as f:
        json.dump(metrics, f, indent=2)

```

```
89 return path
```

Listing 4.2: `src/evaluate.py` — Forecast evaluation metrics and DM test

4.3 src/plots.py

Generates all thesis figures: forecast vs. actual, error distributions, monthly MAE, feature importance, and weather stress index.

```

1 import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import seaborn as sns
import pandas as pd
import numpy as np
6 from src.config import FIGURES_DIR

sns.set_theme(style="whitegrid", palette="muted")
FIGURES_DIR.mkdir(parents=True, exist_ok=True)

11
def plot_price_series(prices: pd.Series, title: str = "Day-Ahead
Prices France", save_as: str = None):
    fig, ax = plt.subplots(figsize=(14, 4))
    ax.plot(prices.index, prices.values, linewidth=0.8)
    ax.set_title(title)
16 ax.set_ylabel("/MWh")
    ax.xaxis.set_major_formatter(mdates.DateFormatter("%b %Y"))
    fig.autofmt_xdate()
    _save(fig, save_as)
    return fig

21
def plot_forecast_vs_actual(y_true: pd.Series, y_pred: pd.Series,
save_as: str = None):
    fig, ax = plt.subplots(figsize=(14, 4))
    ax.plot(y_true.index, y_true.values, label="Actual", linewidth
        =0.9)
26 ax.plot(y_pred.index, y_pred.values, label="Forecast",
        linewidth=0.9, linestyle="--")
    ax.set_title("Forecast vs Actual Day-Ahead Price")
    ax.set_ylabel("/MWh")
    ax.legend()
    _save(fig, save_as)
31 return fig

def plot_feature_importance(model, feature_names: list, top_n: int
= 20, save_as: str = None):
    importances = pd.Series(model.feature_importances_, index=
        feature_names)
36 importances = importances.nlargest(top_n).sort_values()
    fig, ax = plt.subplots(figsize=(8, top_n * 0.35))
    importances.plot(kind="barh", ax=ax)

```



```

    ax.set_title(f"Top {top_n} Feature Importances")
    ax.set_xlabel("Importance")
41  _save(fig, save_as)
    return fig

def plot_error_distribution(y_true: pd.Series, y_pred: pd.Series,
    save_as: str = None):
46  errors = y_pred - y_true
    fig, axes = plt.subplots(1, 2, figsize=(12, 4))
    axes[0].hist(errors, bins=60, edgecolor="white")
    axes[0].set_title("Forecast Error Distribution")
    axes[0].set_xlabel("Error ( /MWh)")
51  axes[1].scatter(y_true, y_pred, alpha=0.3, s=5)
    lims = [min(y_true.min(), y_pred.min()), max(y_true.max(),
        y_pred.max())]
    axes[1].plot(lims, lims, "r--", linewidth=1)
    axes[1].set_title("Actual vs Predicted")
    axes[1].set_xlabel("Actual ( /MWh)")
56  axes[1].set_ylabel("Predicted ( /MWh)")
    _save(fig, save_as)
    return fig

61 def _save(fig: plt.Figure, name: str = None):
    if name:
        fig.tight_layout()
        fig.savefig(FIGURES_DIR / name, dpi=150, bbox_inches="tight")

```

Listing 4.3: src/plots.py — Figure generation

Chapter 5

Trading Backtest

5.1 scripts/backtest.py

Implements the day-ahead directional trading strategy described in Chapter 5 of the thesis.

Key design choices:

- Signal: $\text{sign}(\hat{p}(h, D+1) - p(h, D))$ — executable before 12:00 CET gate closure
- Dead-band filter: $|\hat{p}(h, D+1) - p(h, D)| < 2 \text{ EUR/MWh} \Rightarrow \text{flat}$
- P&L: $\text{signal} \times (p(h, D+1) - p(h, D))$, units EUR/MW
- Transaction costs: 0.10 / 0.30 / 0.60 EUR/MWh per active hour
- Sharpe: daily aggregation, annualised $\times \sqrt{365}$

```
1 """
Phase 3 - Trading Backtest (French Day-Ahead Electricity Market)
=====

DESIGN RATIONALE
6 -----
EPEX SPOT France day-ahead prices are set in a single batch auction
  at noon
(12:00 CET) on day D for all 24 hours of day D+1. Any executable
  strategy
must therefore use only information available before that gate
  closure.

11 Signal (day-ahead correct)
-----
  At noon on day D we know all hourly prices of day D.
  For each hour h of day D+1 we compare our model forecast to the
    same
  hour on day D (the last comparable observation):

16
    signal(h, D+1) = sign( pred(h, D+1) - actual(h, D) )
```

```

+1 = expect the price for hour h tomorrow to be HIGHER than today
    -> LONG
-1 = expect it to be LOWER
                                -> SHORT
21  0 = |pred - ref| < THRESHOLD (dead-band, no trade)

Dead-band: 2 EUR/MWh -- roughly the 20th percentile of |day-on-
    day delta|
on EPEX France. Below this threshold the edge does not cover
    even minimum
    execution costs.

26  P&L (day-on-day, consistent with signal)
    -----

    gross_pnl(h, D+1) = signal(h, D+1) * ( actual(h, D+1) - actual(h,
        D) )

31  Long position profits if tomorrow's price > today's price for the
    same hour.
    Short position profits if tomorrow's price < today's price for
    the same hour.
    Units: EUR per MWh per MW of position (1 MW assumed throughout).

Transaction costs (EPEX SPOT France fee schedule 2024)
36  -----

    Costs apply to EVERY active hour (you submit bids every day for
    each hour
    you want to trade -- not just when you change direction).

    Optimistic  0.10 EUR/MWh  EPEX matching fee only (~0.04) +
        minimal slippage
41  Central      0.30 EUR/MWh  + bid-ask proxy + admin overhead
    Pessimistic 0.60 EUR/MWh  + imbalance settlement risk in low-
        liquidity hours

    net_pnl(h) = gross_pnl(h) - cost * |signal(h)|

46  Risk metrics (methodology)
    -----

    Sharpe : aggregated to DAILY, annualised with sqrt(365) --
        electricity
        trades every calendar day. No risk-free rate (power
        convention).

    Max Drawdown : peak-to-trough on cumulative NET P&L.
51  Calmar      : annualised daily P&L / |MDD|.
    Profit Factor : sum(positive P&L) / |sum(negative P&L)| --
        measures

```

```

consistency of gains vs losses.
Monthly Sharpe: rolling month-by-month net Sharpe -- key
robustness check.
Peak / Off-peak split: hours 8-20 (peak) vs rest (off-peak).
56
Models
-----
A Naive lag-168h (benchmark)
B Random Forest without weather features
61 C Random Forest with weather features (main model)
D XGBoost with weather features
L Long-only reference (always long, buy-and-hold equivalent)

Outputs
66 -----
outputs/metrics/backtest_results.json
outputs/tables/backtest_comparison.csv          central scenario
outputs/tables/backtest_sensitivity.csv         3 scenarios x 4
models
outputs/tables/backtest_monthly_sharpe.csv      monthly Sharpe by
model
71 outputs/figures/equity_curves.png             gross (no costs)
outputs/figures/equity_curves_net.png          net of costs,
central
outputs/figures/monthly_pnl.png
outputs/figures/pnl_distribution.png
outputs/figures/cost_sensitivity.png
76 outputs/figures/rolling_sharpe.png            30-day rolling
Sharpe
"""

import json
import warnings
81 import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
86 import matplotlib.ticker as mticker
from pathlib import Path

warnings.filterwarnings("ignore", category=UserWarning)

91 #
-----

# Paths & constants
#

```

```

-----

BASE_DIR      = Path(__file__).resolve().parents[1]
PREDS_PATH    = BASE_DIR / "outputs" / "predictions" / "
    test_predictions.parquet"
96 METRICS_DIR  = BASE_DIR / "outputs" / "metrics"
TABLES_DIR    = BASE_DIR / "outputs" / "tables"
FIGS_DIR      = BASE_DIR / "outputs" / "figures"

for d in [METRICS_DIR, TABLES_DIR, FIGS_DIR]:
101     d.mkdir(parents=True, exist_ok=True)

COST_SCENARIOS = {
    "optimiste": 0.10,
    "central": 0.30,
106    "pessimiste": 0.60,
}
COST_CENTRAL   = COST_SCENARIOS["central"]
THRESHOLD      = 2.0      # EUR/MWh dead-band
PEAK_HOURS     = range(8, 20) # hours 8-19 inclusive = peak
111 ANN_FACTOR  = 365      # calendar days (electricity market)

PALETTE = {
    "Naif lag-168h": "#9E9E9E",
    "RF sans meteo": "#2196F3",
116    "RF avec meteo": "#4CAF50",
    "XGBoost": "#FF9800",
    "Long-only": "#E91E63",
}

121 #
-----

# Core functions
#
-----

126 def build_reference(actual: pd.Series) -> pd.Series:
    """
    Day-ahead reference price = same hour, previous day (shift 24h)
    .
    This is the last comparable observation available at gate
    closure (noon D)
    for each hour h of D+1.
131    """
    return actual.shift(24)

```

```

def compute_signals(predicted: pd.Series,
136         reference: pd.Series,
        threshold: float = THRESHOLD) -> pd.Series:
    """
    signal(h) = sign(pred(h) - reference(h))
    Dead-band: 0 when |pred - ref| < threshold (no trade).
141    """
    diff = predicted - reference
    signal = np.sign(diff)
    signal[np.abs(diff) < threshold] = 0
    return signal.dropna()
146

def compute_pnl(actual: pd.Series,
        reference: pd.Series,
        signal: pd.Series,
151        cost_per_mwh: float = COST_CENTRAL) -> tuple:
    """
    gross_pnl(h) = signal(h) * (actual(h) - reference(h))
                    = day-on-day price change in the direction of the
                      signal.
    cost(h) = cost_per_mwh * |signal(h)| [per active hour]
156    """
    idx = signal.index
    day_delta = (actual - reference).loc[idx]
    gross_pnl = signal * day_delta
    cost_series = cost_per_mwh * signal.abs()
161    net_pnl = gross_pnl - cost_series
    return gross_pnl.dropna(), net_pnl.dropna()

def sharpe(pnl: pd.Series, ann: int = ANN_FACTOR) -> float:
166    """Annualised Sharpe on daily P&L, sqrt(365), no risk-free rate
        ."""
    daily = pnl.resample("D").sum()
    if daily.std() == 0 or len(daily) < 5:
        return 0.0
    return float(daily.mean() / daily.std() * np.sqrt(ann))
171

def max_dd(cum: pd.Series) -> float:
    return float((cum.cummax() - cum).max())

176
def calmar(pnl: pd.Series, cum: pd.Series, ann: int = ANN_FACTOR)
-> float:

```

```

ann_ret = pnl.resample("D").sum().mean() * ann
mdd      = max_dd(cum)
return float(ann_ret / mdd) if mdd > 0 else np.inf

181

def profit_factor(pnl: pd.Series) -> float:
    pos = pnl[pnl > 0].sum()
    neg = pnl[pnl < 0].abs().sum()
186    return float(pos / neg) if neg > 0 else np.inf

def win_rate(pnl: pd.Series, signal: pd.Series) -> float:
    active = pnl.loc[signal.loc[pnl.index] != 0]
191    return float((active > 0).mean() * 100) if len(active) > 0 else
        0.0

def monthly_sharpe(net_pnl: pd.Series) -> pd.Series:
    """Sharpe computed independently on each calendar month."""
196    results = {}
    for period, grp in net_pnl.groupby(net_pnl.index.to_period("M")):
        daily = grp.resample("D").sum()
        if daily.std() > 0 and len(daily) >= 5:
            results[str(period)] = daily.mean() / daily.std() * np.
                sqrt(ANN_FACTOR)
201        else:
            results[str(period)] = np.nan
    return pd.Series(results)

206 def peak_offpeak_split(pnl: pd.Series, signal: pd.Series) -> dict:
    """Net P&L and win rate separately for peak (8-19) and off-peak
        hours."""
    peak_mask = pnl.index.hour.isin(PEAK_HOURS)
    out = {}
    for name, mask in [("peak", peak_mask), ("offpeak", ~peak_mask)]:
211        p = pnl[mask]
        s = signal.loc[p.index]
        out[name] = {
            "net_pnl": float(p.sum()),
            "win_rate": win_rate(p, s),
216            "n_active": int((s != 0).sum()),
        }
    return out

```

```

221 def run_strategy(actual: pd.Series,
                  reference: pd.Series,
                  predicted: pd.Series,
                  label: str,
                  cost: float = COST_CENTRAL) -> dict:
226     signal = compute_signals(predicted, reference)
    gross_pnl, net_pnl = compute_pnl(actual, reference, signal,
                                    cost)
    cum_gross = gross_pnl.cumsum()
    cum_net = net_pnl.cumsum()

231     total_cost = cost * signal.abs().sum()
    n_active = int((signal != 0).sum())
    cost_drag_pct = (total_cost / cum_gross.iloc[-1] * 100
                     if cum_gross.iloc[-1] != 0 else np.nan)

236     return {
        "label": label,
        "cost_eur_mwh": cost,
        # Gross
        "gross_pnl": float(cum_gross.iloc[-1]),
241     "gross_sharpe": sharpe(gross_pnl),
        # Net
        "net_pnl": float(cum_net.iloc[-1]),
        "net_sharpe": sharpe(net_pnl),
        "net_ann_pnl": float(net_pnl.resample("D").sum().mean
                                   () * ANN_FACTOR),
246     "max_drawdown": max_dd(cum_net),
        "calmar": calmar(net_pnl, cum_net),
        "win_rate": win_rate(net_pnl, signal),
        "profit_factor": profit_factor(net_pnl),
        "n_active_hours": n_active,
251     "active_pct": float(n_active / len(signal) * 100),
        "n_flat_hours": int((signal == 0).sum()),
        "cost_drag_pct": cost_drag_pct,
        "monthly_sharpe": monthly_sharpe(net_pnl),
        "peak_offpeak": peak_offpeak_split(net_pnl, signal),
256     # Series
        "_gross_pnl_s": gross_pnl,
        "_net_pnl_s": net_pnl,
        "_cum_gross": cum_gross,
        "_cum_net": cum_net,
261     "_signal": signal,
    }

#

```



```

266 # Plotting
#
-----

def _ax_setup(ax, title, ylabel):
    ax.set_title(title, fontsize=12, fontweight="bold")
271 ax.set_ylabel(ylabel)
    ax.grid(True, alpha=0.3)
    ax.yaxis.set_major_formatter(
        mticker.FuncFormatter(lambda x, _: f"{x:,.0f}"))

276 def plot_equity(strats: dict, net: bool, suffix: str):
    key = "_cum_net" if net else "_cum_gross"
    cost_note = f" (frais : {COST_CENTRAL} EUR/MWh/h active)" if
        net else " (sans frais)"
    title = ("Performance cumulee NETTE" if net else "Performance
        cumulee BRUTE") + cost_note

281 fig, ax = plt.subplots(figsize=(13, 6))
    for k, s in strats.items():
        c = PALETTE.get(s["label"], "#333")
        lw = 2.5 if "RF avec" in s["label"] else 1.5
286 ls = "--" if s["label"] == "Long-only" else "-"
        ax.plot(s[key].index, s[key].values,
                label=s["label"], color=c, linewidth=lw, linestyle=
                    ls)
        ax.axhline(0, color="black", linewidth=0.8, linestyle=":")
        _ax_setup(ax, title, "P&L cumule (EUR/MW)")
291 ax.set_xlabel("Date")
        ax.legend(loc="upper left", fontsize=10)
        plt.tight_layout()
        p = FIGS_DIR / f"equity_curves{suffix}.png"
        fig.savefig(p, dpi=150, bbox_inches="tight")
296 plt.close(fig)
        print(f" Saved: {p}")

def plot_monthly_pnl(strats: dict):
301 keys = [k for k in strats if strats[k]["label"] in
            ("RF avec meteo", "Naif lag-168h", "XGBoost")]
    fig, ax = plt.subplots(figsize=(14, 5))
    bw = 0.25
    months = None
306 for i, key in enumerate(keys):
        s = strats[key]

```

```

    mp = s["_net_pnl_s"].resample("ME").sum()
    mp.index = mp.index.to_period("M").to_timestamp()
    if months is None:
311         months = mp.index
    x = np.arange(len(mp))
    ax.bar(x + (i - len(keys))/2 + 0.5) * bw, mp.values, bw,
          label=s["label"], color=PALETTE.get(s["label"], "#888"), alpha=0.85)
    if months is not None:
316         ax.set_xticks(np.arange(len(months)))
        ax.set_xticklabels([m.strftime("%b %Y") for m in months],
                           rotation=45, ha="right", fontsize=8)
    ax.axhline(0, color="black", linewidth=0.8)
    ax.set_title(f"P&L mensuel NET (EUR/MW) - scenario central ({
        COST_CENTRAL} EUR/MWh)",
321                 fontsize=12, fontweight="bold")
    ax.set_ylabel("P&L (EUR/MW)")
    ax.yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _ :
        f"{x:,.0f}"))
    ax.legend(fontsize=10)
    ax.grid(True, alpha=0.3, axis="y")
326 plt.tight_layout()
    p = FIGS_DIR / "monthly_pnl.png"
    fig.savefig(p, dpi=150, bbox_inches="tight")
    plt.close(fig)
    print(f" Saved: {p}")
331

def plot_pnl_distribution(strats: dict):
    labels = [s["label"] for s in strats.values()]
    data = [s["_net_pnl_s"].values for s in strats.values()]
336 colors = [PALETTE.get(l, "#888") for l in labels]
    fig, ax = plt.subplots(figsize=(11, 5))
    bp = ax.boxplot(data, patch_artist=True, tick_labels=labels,
                    whis=1.5, showfliers=False,
                    medianprops={"color": "black", "linewidth": 2})
341 for patch, color in zip(bp["boxes"], colors):
        patch.set_facecolor(color); patch.set_alpha(0.7)
    ax.axhline(0, color="black", linewidth=0.8, linestyle=":")
    ax.set_title(f"Distribution P&L journalier NET (EUR/MW) -
        central ({COST_CENTRAL} EUR/MWh)",
346                 fontsize=12, fontweight="bold")
    ax.set_ylabel("P&L (EUR/MW)")
    ax.grid(True, alpha=0.3, axis="y")
    plt.xticks(rotation=15, ha="right")
    plt.tight_layout()
    p = FIGS_DIR / "pnl_distribution.png"
351 fig.savefig(p, dpi=150, bbox_inches="tight")

```

```

plt.close(fig)
print(f"  Saved: {p}")

356 def plot_cost_sensitivity(sens: pd.DataFrame):
    models      = [m for m in sens["Modele"].unique() if "Long" not
                    in m]
    scenarios    = list(COST_SCENARIOS.keys())
    x            = np.arange(len(models))
    w            = 0.25
361 fig, axes = plt.subplots(1, 2, figsize=(14, 5))
    for ax, col, ylabel, title in [
        (axes[0], "P&L net (EUR/MW)",      "P&L net (EUR/MW)",      "P&
          L net par scenario de frais"),
        (axes[1], "Sharpe net (daily)",    "Sharpe annualise",      "
          Sharpe net par scenario de frais"),
    ]:
366         for i, sc in enumerate(scenarios):
            sub = sens[sens["Scenario"] == sc]
            vals = [sub[sub["Modele"] == m][col].values[0]
                    if len(sub[sub["Modele"] == m]) > 0 else 0
                    for m in models]
371             ax.bar(x + (i-1)*w, vals, w,
                    label=f"{sc} ({COST_SCENARIOS[sc]} EUR/MWh)",
                    alpha=0.85)
            ax.set_xticks(x)
            ax.set_xticklabels(models, rotation=15, ha="right",
                               fontsize=9)
            ax.set_title(title, fontsize=11, fontweight="bold")
376             ax.set_ylabel(ylabel)
            ax.axhline(0, color="black", linewidth=0.7)
            ax.legend(fontsize=9)
            ax.grid(True, alpha=0.3, axis="y")
            fig.suptitle("Analyse de sensibilite aux couts de transaction",
381                        fontsize=13, fontweight="bold")
            plt.tight_layout()
            p = FIGS_DIR / "cost_sensitivity.png"
            fig.savefig(p, dpi=150, bbox_inches="tight")
            plt.close(fig)
386             print(f"  Saved: {p}")

def plot_rolling_sharpe(strats: dict, window_days: int = 30):
    """30-day rolling Sharpe on net P&L      shows temporal
        stability."""
391     fig, ax = plt.subplots(figsize=(13, 5))
    for key, s in strats.items():
        if s["label"] == "Long-only":

```

```

        continue
        daily = s["_net_pnl_s"].resample("D").sum()
396         roll = daily.rolling(window_days, min_periods=15)
        rs = roll.mean() / roll.std() * np.sqrt(ANN_FACTOR)
        ax.plot(rs.index, rs.values,
                label=s["label"],
                color=PALETTE.get(s["label"], "#333"),
401                linewidth=1.6)
        ax.axhline(0, color="black", linewidth=0.8, linestyle=":")
        ax.axhline(2, color="grey", linewidth=0.8, linestyle="--",
                alpha=0.6)
        ax.text(ax.get_xlim()[0], 2.1, "Sharpe = 2 (ref. hedge fund)",
                fontsize=8, color="grey")
406        ax.set_title(f"Sharpe glissant {window_days}j (NET, annualise
                sqrt(365))",
                fontsize=12, fontweight="bold")
        ax.set_ylabel("Sharpe")
        ax.set_xlabel("Date")
        ax.legend(fontsize=10)
411        ax.grid(True, alpha=0.3)
        plt.tight_layout()
        p = FIGS_DIR / "rolling_sharpe.png"
        fig.savefig(p, dpi=150, bbox_inches="tight")
        plt.close(fig)
416        print(f" Saved: {p}")

#
-----

# Main
421 #
-----

def main():
    print("=" * 65)
    print("FR Power Thesis -- Phase 3: Trading Backtest (day-ahead)
        ")
    print("=" * 65)
426
    # 1. Load predictions
    print(f"\n[1/6] Loading {PREDS_PATH}...")
    if not PREDS_PATH.exists():
        raise FileNotFoundError(
431            f"{PREDS_PATH} not found.\nRun scripts/
                run_full_pipeline.py first.")
    df = pd.read_parquet(PREDS_PATH)
    print(f" {len(df):,} obs | {df.index.min().date()} -> {df.

```

```

        index.max().date()})")

actual      = df["actual"]
436 reference = build_reference(actual)    # same hour, day D (shift
        24h)

# Drop first 24 hours (reference = NaN)
valid_idx = reference.dropna().index
actual     = actual.loc[valid_idx]
441 reference = reference.loc[valid_idx]
df         = df.loc[valid_idx]

print(f"  After 24h warm-up drop: {len(actual):,} obs "
      f"({actual.index.min().date()} -> {actual.index.max().
        date()})")
446 print(f"\n  Signal design: sign(pred(h,D+1) - actual(h,D))  [
        day-ahead executable]")
print(f"  P&L design      : signal * (actual(h,D+1) - actual(h,D))
        ")
print(f"  Dead-band      : {THRESHOLD} EUR/MWh")
print(f"  Cost model      : {COST_CENTRAL} EUR/MWh per active hour
        (central scenario)")
print(f"  Annualisation: sqrt({ANN_FACTOR}) (calendar days)")
451

model_map = {
    "naive":          ("Naif lag-168h", df["naive"]),
    "rf_no_weather": ("RF sans meteo", df["rf_no_weather"]),
    "rf_weather":     ("RF avec meteo", df["rf_weather"]),
456    "xgboost":       ("XGBoost",      df["xgboost"]),
}

# 2. Central scenario
print(f"\n[2/6] Central scenario (cost = {COST_CENTRAL} EUR/MWh
        )...")
461 strats = {}
for key, (label, pred) in model_map.items():
    strats[key] = run_strategy(actual, reference, pred, label,
        COST_CENTRAL)

# Long-only reference
466 lo_pnl = (actual - reference).dropna()
lo_cum  = lo_pnl.cumsum()
lo_sig  = pd.Series(1.0, index=lo_pnl.index)
strats["long_only"] = {
    "label": "Long-only", "cost_eur_mwh": 0.0,
471    "gross_pnl": float(lo_cum.iloc[-1]), "gross_sharpe": sharpe
        (lo_pnl),
    "net_pnl": float(lo_cum.iloc[-1]), "net_sharpe": sharpe

```

```

        (lo_pnl),
        "net_ann_pnl": float(lo_pnl.resample("D").sum().mean() *
            ANN_FACTOR),
        "max_drawdown": max_dd(lo_cum), "calmar": calmar(lo_pnl,
            lo_cum),
        "win_rate": win_rate(lo_pnl, lo_sig),
476     "profit_factor": profit_factor(lo_pnl),
        "n_active_hours": len(lo_pnl), "active_pct": 100.0,
        "n_flat_hours": 0, "cost_drag_pct": 0.0,
        "monthly_sharpe": monthly_sharpe(lo_pnl),
        "peak_offpeak": peak_offpeak_split(lo_pnl, lo_sig),
481     "_gross_pnl_s": lo_pnl, "_net_pnl_s": lo_pnl,
        "_cum_gross": lo_cum, "_cum_net": lo_cum,
        "_signal": lo_sig,
    }

# Print summary table
486     print(f"\n  {'Modele':<22} {'GrossP&L':>9} {'NetP&L':>9} "
            f"{'Sharpe(net)':>12} {'MDD':>8} {'PF':>6} "
            f"{'Win%':>6} {'Active%':>8} {'CostDrag':>9}")
    print("  " + "-" * 96)
491     for key in ["naive", "rf_no_weather", "rf_weather", "xgboost",
        "long_only"]:
        s = strats[key]
        print(f"  {s['label']:<22} "
            f"{s['gross_pnl']:>9,.0f} "
            f"{s['net_pnl']:>9,.0f} "
496         f"{s['net_sharpe']:>12.3f} "
            f"{s['max_drawdown']:>8,.0f} "
            f"{s['profit_factor']:>6.2f} "
            f"{s['win_rate']:>5.1f}% "
            f"{s['active_pct']:>7.1f}% "
501         f"{s['cost_drag_pct']:>8.1f}%")

# Peak / off-peak
    print(f"\n  Peak vs Off-Peak NET P&L (central scenario):")
    print(f"  {'Modele':<22} {'Peak P&L':>10} {'Peak Win%':>10} "
506         f"{'OffPk P&L':>10} {'OffPk Win%':>11}")
    print("  " + "-" * 66)
    for key in ["naive", "rf_no_weather", "rf_weather", "xgboost"]:
        s = strats[key]
        pk = s["peak_offpeak"]["peak"]
511         op = s["peak_offpeak"]["offpeak"]
        print(f"  {s['label']:<22} "
            f"{pk['net_pnl']:>10,.0f} "
            f"{pk['win_rate']:>9.1f}% "
            f"{op['net_pnl']:>10,.0f} "
516         f"{op['win_rate']:>10.1f}%")

```

```

# Monthly Sharpe
print(f"\n Sharpe mensuel NET (scenario central) :")
ms_df = pd.DataFrame({
521     strats[k]["label"]: strats[k]["monthly_sharpe"]
    for k in ["naive", "rf_no_weather", "rf_weather", "xgboost"
    ]
}).round(2)
print(" " + ms_df.to_string().replace("\n", "\n "))

# 3. Sensitivity analysis
526 print(f"\n[3/6] Sensitivity analysis ({len(COST_SCENARIOS)}
    scenarios)...")
sens_rows = []
for sc, cost in COST_SCENARIOS.items():
    for key, (label, pred) in model_map.items():
531         s = run_strategy(actual, reference, pred, label, cost)
        sens_rows.append({
            "Scenario":          sc,
            "Cout (EUR/MWh)":    cost,
            "Modele":            label,
536             "P&L brut (EUR/MW)": round(s["gross_pnl"], 0),
            "P&L net (EUR/MW)":  round(s["net_pnl"], 0),
            "Sharpe brut":       round(s["gross_sharpe"], 2),
            "Sharpe net (daily)": round(s["net_sharpe"], 2),
            "Max Drawdown":      round(s["max_drawdown"], 0),
541             "Calmar net":       round(s["calmar"], 2),
            "Win Rate (%)":      round(s["win_rate"], 1),
            "Profit Factor":     round(s["profit_factor"], 2),
            "Heures actives":    s["n_active_hours"],
            "Cost Drag (%)":     round(s["cost_drag_pct"], 1),
546         })
sens_df = pd.DataFrame(sens_rows)
sens_df.to_csv(TABLES_DIR / "backtest_sensitivity.csv", index=
    False)
print(f" Saved: {TABLES_DIR / 'backtest_sensitivity.csv'}")

551 pivot_sharpe = sens_df.pivot_table(
    index="Modele", columns="Scenario",
    values="Sharpe net (daily)", aggfunc="first"
)[list(COST_SCENARIOS.keys())]
print("\n Sharpe NET par scenario :")
556 print(" " + pivot_sharpe.round(2).to_string().replace("\n", "\n
    "))

# 4. Monthly Sharpe table
ms_df.to_csv(TABLES_DIR / "backtest_monthly_sharpe.csv")
print(f"\n Saved: {TABLES_DIR / 'backtest_monthly_sharpe.csv'}")

```

```

    ")

561
    # 5. Save JSON + CSV
    print("\n[4/6] Saving outputs...")
    skip = {"_gross_pnl_s", "_net_pnl_s", "_cum_gross",
            "_cum_net", "_signal", "monthly_sharpe", "peak_offpeak"
            }
566
    json_out = {}
    for key, s in strats.items():
        json_out[key] = {k: (round(v, 4) if isinstance(v, float)
                             else v)
                        for k, v in s.items() if k not in skip}
    with open(METRICS_DIR / "backtest_results.json", "w") as f:
571
        json.dump(json_out, f, indent=2)
    print(f" Saved: {METRICS_DIR / 'backtest_results.json'}")

    col_map = {
        "label": "Modele",
576
        "gross_pnl": "P&L brut (EUR/MW)",
        "net_pnl": "P&L net (EUR/MW)",
        "net_sharpe": "Sharpe net (daily,sqrt365)",
        "max_drawdown": "Max Drawdown (EUR/MW)",
        "calmar": "Calmar",
581
        "win_rate": "Win Rate (%)",
        "profit_factor": "Profit Factor",
        "n_active_hours": "Heures actives",
        "active_pct": "Actif (%)",
        "cost_drag_pct": "Cost Drag (%)",
586
    }
    rows = []
    for key in ["naive", "rf_no_weather", "rf_weather", "xgboost",
               "long_only"]:
        s = strats[key]
        rows.append({v: (round(s[k], 2) if isinstance(s[k], float)
                         else s[k])
                    for k, v in col_map.items()})
591
    comp_df = pd.DataFrame(rows)
    comp_df.to_csv(TABLES_DIR / "backtest_comparison.csv", index=
                  False)
    print(f" Saved: {TABLES_DIR / 'backtest_comparison.csv'}")
    print()
596
    print(comp_df.to_string(index=False))

    # 6. Plots
    print("\n[5/6] Generating plots...")
    plot_equity(strats, net=False, suffix="")
    plot_equity(strats, net=True, suffix="_net")
601
    plot_monthly_pnl(strats)

```



```

plot_pnl_distribution(strats)
plot_cost_sensitivity(sens_df)
plot_rolling_sharpe(strats)

606

print("\n[6/6] Done. Summary:")
print(f"   Signal       : sign(pred(h,D+1) - actual(h,D))   [day-
      ahead executable]")
print(f"   P&L           : signal * (actual(h,D+1) - actual(h,D))")
print(f"   Dead-band      : {THRESHOLD} EUR/MWh")
611 print(f"   Cost          : {COST_CENTRAL} EUR/MWh per active hour (
      central)")
print(f"   Ann.           : sqrt({ANN_FACTOR}) on daily aggregation")
print(f"\n   NOTE ON SHARPE: Sharpe > 2 reflects the strategy's
      directional")
print(f"   consistency over this specific test period. It is NOT
      comparable")
print(f"   to equity fund Sharpe ratios (different asset class,
      no leverage,")
616 print(f"   different cost structure). The monthly Sharpe table
      shows whether")
print(f"   performance is robust across months or driven by a
      few outliers.")
print(f"\nAll outputs saved to outputs/")

621 if __name__ == "__main__":
    main()

```

Listing 5.1: scripts/backtest.py — Day-ahead directional trading backtest

Chapter 6

2022 Energy Crisis Validation

6.1 scripts/test_2022.py

Retrains Models A–D on 2018–2021 data and evaluates them on the full year 2022 (8,760 hours). Produces the performance table and DM tests for the 2022 crisis regime reported in Section 4.7 of the thesis. The key result: weather features are highly significant in 2022 (DM = -13.27 , $p < 0.001$) versus non-significant in the stable 2024–25 period ($p = 0.572$).

```
"""
Robustness check: out-of-sample validation on the 2022 energy
    crisis year.
3
Rationale
-----
The main pipeline tests on May 2024      April 2025, a period of
    relatively
stable prices after the energy crisis.  A referee could argue that
    results
8 are overly optimistic because the test window avoids the most
    turbulent
period.  This script addresses that critique directly:

    Train : 2018-01-08 -> 2021-12-31  (4 years, pre-crisis)
    Test  : 2022-01-01 -> 2022-12-31  (full crisis year)
13
2022 is the most informative stress test for the French market:
- EDF nuclear fleet at ~40 % availability (corrosion issues)
- TTF gas prices 10    historical average (Aug 2022 peak: 339 EUR/
    MWh)
- Day-ahead prices spiked above 1 000 EUR/MWh multiple times
18 - Extreme price volatility unlike any post-2010 year

Models trained identically to the main pipeline (same
    hyperparameters,
same feature set).  Results saved alongside the main metrics so
    both
```

```

periods can be reported in the thesis Results section.
23
Outputs
-----
    outputs/metrics/validation_2022_A.json    (naive)
    outputs/metrics/validation_2022_B.json    (RF sans meteo)
28    outputs/metrics/validation_2022_C.json    (RF avec meteo)
    outputs/metrics/validation_2022_D.json    (XGBoost)
    outputs/metrics/dm_tests_2022.json
    outputs/tables/model_comparison_2022.csv
    """
33 import json
import sys
import pandas as pd
from pathlib import Path

38 ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT))

from src.config import PROCESSED_DIR, TABLES_DIR, METRICS_DIR
from src.features import build_feature_matrix, WEATHER_FEATURE_COLS
43 from src.models import train_random_forest, train_xgboost,
    naive_forecast
from src.evaluate import compute_all, dm_test

TABLES_DIR.mkdir(parents=True, exist_ok=True)
METRICS_DIR.mkdir(parents=True, exist_ok=True)
48
TRAIN_END = "2021-12-31 23:00:00"
TEST_START = "2022-01-01 00:00:00"
TEST_END   = "2022-12-31 23:00:00"

53
def main():
    print("=" * 60)
    print("FR Power Thesis      2022 Crisis Robustness Validation")
    print("=" * 60)
58
    #
    -----

    # 1. Load data & build feature matrices
    #
    -----

    print("\n[1/4] Loading processed dataset...")
63    df = pd.read_parquet(PROCESSED_DIR / "features.parquet")
    print(f"    Full dataset: {df.index.min().date()} -> {df.index.

```

```

max().date()})")

print("\n[2/4] Building feature matrices...")
X, y = build_feature_matrix(df, include_weather=
    True)
68 X_no_wx, y_no_wx = build_feature_matrix(df, include_weather=
    False)

#
-----

# 2. 2022 train/test split
#
-----

73 # Timezone-aware slicing      handle both tz-aware and tz-naive
    indices
def tz_slice(series, start, end):
    idx = series.index
    if idx.tz is not None:
        start = pd.Timestamp(start).tz_localize(idx.tz)
        end = pd.Timestamp(end).tz_localize(idx.tz)
78         train_end_ts = pd.Timestamp(TRAIN_END).tz_localize(idx.
            tz)
    else:
        start = pd.Timestamp(start)
        end = pd.Timestamp(end)
83         train_end_ts = pd.Timestamp(TRAIN_END)
    train = series.loc[:train_end_ts]
    test = series.loc[start:end]
    return train, test

88 X_train, X_test = tz_slice(X, TEST_START, TEST_END)
    y_train, y_test = tz_slice(y, TEST_START, TEST_END
    )
    X_b_train, X_b_test = tz_slice(X_no_wx, TEST_START, TEST_END)
    y_b_train, y_b_test = tz_slice(y_no_wx, TEST_START, TEST_END)

93 print(f" Train: {len(X_train):,} hours ({X_train.index.min().
    date()} -> {X_train.index.max().date()})")
print(f" Test : {len(X_test):,} hours ({X_test.index.min().
    date()} -> {X_test.index.max().date()})")

#
-----

# 3. Train models
98 #

```

```

-----

print("\n[3/4] Training models on pre-crisis data (2018 - 2021 )
    ...")

print("  Mod le A      Naive (lag-168h)...")
# Naive uses the full y series to look back 168h from test
  start
103 y_pred_naive = naive_forecast(y).loc[y_test.index]

print("  Mod le B      Random Forest WITHOUT weather...")
rf_b = train_random_forest(X_b_train, y_b_train)
y_pred_rf_b = pd.Series(rf_b.predict(X_b_test), index=y_b_test.
    index)
108

print("  Mod le C      Random Forest WITH weather...")
rf_c = train_random_forest(X_train, y_train)
y_pred_rf_c = pd.Series(rf_c.predict(X_test), index=y_test.
    index)

113 print("  Mod le D      XGBoost WITH weather...")
xgb = train_xgboost(X_train, y_train)
y_pred_xgb = pd.Series(xgb.predict(X_test), index=y_test.index)

# Align to common index
118 common_idx = y_test.index
for pred in [y_pred_naive, y_pred_rf_b, y_pred_rf_c, y_pred_xgb
    ]:
    common_idx = common_idx.intersection(pred.dropna().index)

y_true      = y_test.loc[common_idx]
123 naive_pred = y_pred_naive.loc[common_idx]
rf_b_pred   = y_pred_rf_b.loc[common_idx]
rf_c_pred   = y_pred_rf_c.loc[common_idx]
xgb_pred    = y_pred_xgb.loc[common_idx]

128 #
-----

# 4. Evaluate & save
#
-----

print("\n[4/4] Evaluating models on 2022 crisis test set...")

133 results = {
    "A_naive":      compute_all(y_true, naive_pred),
    "B_rf_no_wx":   compute_all(y_true, rf_b_pred),

```

```

    "C_rf_wx":      compute_all(y_true, rf_c_pred),
    "D_xgboost":    compute_all(y_true, xgb_pred),
138 }

print(f"\n{'Model':<22} {'MAE':>8} {'RMSE':>8} {'sMAPE':>8} {'R'
      ' ':>8} {'Hit%':>7}")
print("-" * 60)
for name, m in results.items():
143     print(f"    {name:<20} {m['mae']:>7.2f} {m['rmse']:>8.2f} "
          f"{m['smape']:>7.1f}% {m['r2']:>7.3f} {m['hit_ratio'
              '']:>6.1f}%")

# Save individual JSON metrics
label_map = {
148     "A_naive":      "validation_2022_A",
    "B_rf_no_wx":    "validation_2022_B",
    "C_rf_wx":       "validation_2022_C",
    "D_xgboost":     "validation_2022_D",
}
153 for key, fname in label_map.items():
    path = METRICS_DIR / f"{fname}.json"
    with open(path, "w") as f:
        json.dump(results[key], f, indent=2)
    print(f"    Saved: {path}")

158 # DM tests
print("\n    DM tests (2022 crisis period):")
dm_results = {}
pairs = [
163     ("C_vs_B_2022", rf_c_pred, rf_b_pred, "Weather vs no-
        weather (C vs B)"),
    ("C_vs_A_2022", rf_c_pred, naive_pred, "RF-weather vs naive
        (C vs A)"),
    ("B_vs_A_2022", rf_b_pred, naive_pred, "RF-no-weather vs
        naive (B vs A)"),
    ("D_vs_C_2022", xgb_pred, rf_c_pred, "XGBoost vs RF-
        weather (D vs C)"),
]
168 print(f"    {'Test':<38} {'DM stat':>9} {'p-value':>9} {'Signif
        .':>9}")
print("    " + "-" * 68)
for key, p1, p2, label in pairs:
    stat, pval = dm_test(y_true, p1, p2)
    sig = "***" if pval < 0.01 else ("**" if pval < 0.05 else (
        "*" if pval < 0.10 else "n.s."))
173 dm_results[key] = {
    "description": label,
    "dm_stat": round(stat, 4),

```

```

        "p_value": round(pval, 4),
        "significant_5pct": pval < 0.05,
178     }
    print(f"  {label:<38} {stat:>9.3f} {pval:>9.4f} {sig:>9}")

    dm_path = METRICS_DIR / "dm_tests_2022.json"
    with open(dm_path, "w") as f:
183         json.dump(dm_results, f, indent=2)
    print(f"\n  DM-test 2022 results -> {dm_path}")

    # Comparison table
    model_labels = {
188         "A_naive":      "Naif (lag-168h)",
        "B_rf_no_wx":    "Random Forest (sans meteo)",
        "C_rf_wx":       "Random Forest (avec meteo)",
        "D_xgboost":     "XGBoost (avec meteo)",
    }

193     rows = []
    for key, label in model_labels.items():
        m = results[key]
        rows.append({
198             "Model":      label,
            "MAE (EUR/MWh)": round(m["mae"], 3),
            "RMSE (EUR/MWh)": round(m["rmse"], 3),
            "sMAPE (%)":     round(m["smape"], 3),
            "R  ":           round(m["r2"], 3),
            "Hit ratio (%)": round(m["hit_ratio"], 1),
203         })
    comp_df = pd.DataFrame(rows)
    csv_path = TABLES_DIR / "model_comparison_2022.csv"
    comp_df.to_csv(csv_path, index=False)
    print(f"\nComparison table (2022) -> {csv_path}")
208     print(comp_df.to_string(index=False))

    print("\nOK 2022 crisis robustness validation complete.")

213 if __name__ == "__main__":
    main()

```

Listing 6.1: scripts/test_2022.py — 2022 energy crisis out-of-sample evaluation

Chapter 7

End-to-End Pipeline

7.1 scripts/run_full_pipeline.py

Master orchestration script that runs the complete pipeline in sequence: data loading, feature engineering, model training (Models A–D), evaluation, figure generation, and backtest. Running this script from the project root reproduces all results in the thesis.

```
1  """
    End-to-end pipeline: load processed data -> train 4 models ->
        evaluate -> DM-test -> save outputs.

    Models trained:
        A      Naive (benchmark) : price(h) = price(h-168), same hour
                previous week
6       B      Random Forest WITHOUT weather features
        C      Random Forest WITH weather features          main model
        D      XGBoost WITH weather features

    Statistical tests:
11      DM-test (Diebold-Mariano, 1995) with Harvey-Leybourne-Newbold
        correction:
        C vs B : does adding weather significantly improve over RF
                baseline?
        C vs A : does RF-weather beat naive?
        B vs A : does RF (no weather) beat naive?
    """
16  import json
    import pandas as pd
    from pathlib import Path
    from src.config import PROCESSED_DIR, TABLES_DIR, METRICS_DIR,
        TEST_MONTHS
    from src.features import build_feature_matrix, WEATHER_FEATURE_COLS
21  from src.models import train_random_forest, train_xgboost,
        naive_forecast
    from src.evaluate import compute_all, save_metrics, dm_test
    from src.plots import plot_forecast_vs_actual,
```



```

    plot_feature_importance

TABLES_DIR.mkdir(parents=True, exist_ok=True)
METRICS_DIR.mkdir(parents=True, exist_ok=True)

def main():
    print("=" * 60)
    print("FR Power Thesis      Full Training Pipeline")
    print("=" * 60)

    #
    -----

    # 1. Load data & build feature matrices
    #
    -----

    print("\n[1/5] Loading processed dataset...")
    df = pd.read_parquet(PROCESSED_DIR / "features.parquet")
    print(f"   {len(df):,} rows, {df.shape[1]} columns")
    print(f"   Period: {df.index.min()} -> {df.index.max()}")

    print("\n[2/5] Building feature matrices...")

    # Mod le C / D      with full weather features
    X, y = build_feature_matrix(df, include_weather=True)
    print(f"   X_weather      : {X.shape[0]:,} rows      {X.shape[1]}
        features")

    # Mod le B          without weather (drop ERA5 and derived weather
        cols)
    X_no_wx, y_no_wx = build_feature_matrix(df, include_weather=
        False)
    print(f"   X_no_weather: {X_no_wx.shape[0]:,} rows      {X_no_wx.
        shape[1]} features")

    weather_features_used = [c for c in WEATHER_FEATURE_COLS if c
        in X.columns]
    print(f"   Weather cols dropped for Mod le B: {
        weather_features_used}")

    #
    -----

    # 2. Train/test split (last TEST_MONTHS months = out-of-sample)
    #
    -----

```

```

cutoff = len(X) - TEST_MONTHS * 30 * 24
X_train, X_test = X.iloc[:cutoff], X.iloc[cutoff:]
y_train, y_test = y.iloc[:cutoff], y.iloc[cutoff:]

61
cutoff_b = len(X_no_wx) - TEST_MONTHS * 30 * 24
X_b_train, X_b_test = X_no_wx.iloc[:cutoff_b], X_no_wx.iloc[
    cutoff_b:]
y_b_train, y_b_test = y_no_wx.iloc[:cutoff_b], y_no_wx.iloc[
    cutoff_b:]

66
print(f"\n Train: {len(X_train):,} hours | Test: {len(X_test)
    :,} hours")
print(f" Test period: {y_test.index.min()} -> {y_test.index.
    max()}")

#
-----

# 3. Train models
#
71
-----

print("\n[3/5] Training models...")

print(" Mod le A Naive (lag-168h)...")
y_pred_naive = naive_forecast(y).loc[y_test.index]

76
print(" Mod le B Random Forest WITHOUT weather...")
rf_b = train_random_forest(X_b_train, y_b_train)
y_pred_rf_b = pd.Series(rf_b.predict(X_b_test), index=y_b_test.
    index)

81
print(" Mod le C Random Forest WITH weather...")
rf_c = train_random_forest(X_train, y_train)
y_pred_rf_c = pd.Series(rf_c.predict(X_test), index=y_test.
    index)

86
print(" Mod le D XGBoost WITH weather...")
xgb = train_xgboost(X_train, y_train)
y_pred_xgb = pd.Series(xgb.predict(X_test), index=y_test.index)

# Align all predictions to the same test index (intersection)
common_idx = y_test.index
91
for pred in [y_pred_naive, y_pred_rf_b, y_pred_rf_c, y_pred_xgb
]:
    common_idx = common_idx.intersection(pred.dropna().index)

```

```

y_true      = y_test.loc[common_idx]
naive_pred  = y_pred_naive.loc[common_idx]
96 rf_b_pred   = y_pred_rf_b.loc[common_idx]
rf_c_pred   = y_pred_rf_c.loc[common_idx]
xgb_pred    = y_pred_xgb.loc[common_idx]

#
-----

101 # 4. Evaluate models
#
-----

print("\n[4/5] Evaluating models...")

results = {
106     "naive":      compute_all(y_true, naive_pred),
     "rf_no_weather": compute_all(y_true, rf_b_pred),
     "rf_weather":  compute_all(y_true, rf_c_pred),
     "xgboost":     compute_all(y_true, xgb_pred),
}

111 print(f"\n{'Model':<20} {'MAE':>8} {'RMSE':>8} {'sMAPE':>8} {'R'
      ' ':>8} {'Hit%':>7}")
print("-" * 60)
for name, m in results.items():
    print(
116         f"    {name:<18} {m['mae']:>7.2f} {m['rmse']:>8.2f} "
         f"    {m['smape']:>7.1f}% {m['r2']:>7.3f} {m['hit_ratio'
              '']:>6.1f}%"
    )

for name, m in results.items():
121     save_metrics(m, name)

#
-----

# 5. Diebold-Mariano tests
#
-----

126 print("\n[5/5] Diebold-Mariano tests...")

dm_results = {}

pairs = [
131     ("rf_weather_vs_rf_no_weather", rf_c_pred, rf_b_pred,

```

```

    "Does weather add value? (C vs B)",
    ("rf_weather_vs_naive",          rf_c_pred, naive_pred,
     "Does RF-weather beat naive? (C vs A)",
    ("rf_no_weather_vs_naive",      rf_b_pred, naive_pred,
136     "Does RF-no-weather beat naive? (B vs A)",
    ("xgboost_vs_rf_weather",      xgb_pred,  rf_c_pred,
     "XGBoost vs RF-weather (D vs C)",
    ]

141 print(f"\n  {'Test':<40} {'DM stat':>9} {'p-value':>9} {'Signif
    .':>9}")
print("  " + "-" * 70)
for key, pred1, pred2, label in pairs:
    stat, pval = dm_test(y_true, pred1, pred2)
    sig = "***" if pval < 0.01 else ("**" if pval < 0.05 else (
        "*" if pval < 0.10 else "n.s."))
146 dm_results[key] = {
    "description": label,
    "dm_stat": round(stat, 4),
    "p_value": round(pval, 4),
    "significant_5pct": pval < 0.05,
151 }
    print(f"  {label:<40} {stat:>9.3f} {pval:>9.4f} {sig:>9}")

dm_path = METRICS_DIR / "dm_tests.json"
with open(dm_path, "w") as f:
156     json.dump(dm_results, f, indent=2)
print(f"\n  DM-test results saved -> {dm_path}")

#
-----

# 6. Save comparison table
161 #
-----

model_labels = {
    "naive":          "Na f (lag-168h)",
    "rf_no_weather": "Random Forest (sans m t o)",
    "rf_weather":    "Random Forest (avec m t o)",
166     "xgboost":      "XGBoost (avec m t o)",
}
rows = []
for key, label in model_labels.items():
    m = results[key]
171     rows.append({
        "Model": label,
        "MAE ( /MWh)": round(m["mae"], 3),

```

```

    "RMSE ( /MWh)": round(m["rmse"], 3),
    "sMAPE (%)": round(m["smape"], 3),
176     "R " : round(m["r2"], 3),
    "Hit ratio (%)": round(m["hit_ratio"], 1),
    })
comp_df = pd.DataFrame(rows)
comp_path = TABLES_DIR / "model_comparison.csv"
181 comp_df.to_csv(comp_path, index=False)
print(f"\nModel comparison table saved -> {comp_path}")
print(comp_df.to_string(index=False))

#
-----

186 # 7. Save test predictions (used by backtest.py)
#
-----

PREDS_DIR = METRICS_DIR.parent / "predictions"
PREDS_DIR.mkdir(parents=True, exist_ok=True)
preds_df = pd.DataFrame({
191     "actual": y_true,
    "naive": naive_pred,
    "rf_no_weather": rf_b_pred,
    "rf_weather": rf_c_pred,
    "xgboost": xgb_pred,
196 })
preds_df.to_parquet(PREDS_DIR / "test_predictions.parquet")
print(f"\nTest predictions saved -> {PREDS_DIR / '
    test_predictions.parquet'}")

#
-----

201 # 8. Plots
#
-----

print("\nGenerating plots...")
try:
    plot_forecast_vs_actual(y_true, rf_c_pred, save_as="
        forecast_vs_actual.png")
    plot_feature_importance(rf_c, X_train.columns.tolist(),
206         save_as="feature_importance_rf.png")
    print(" Plots saved to outputs/figures/")
except Exception as e:
    print(f" WARNING: plot generation failed: {e}")

```

```
211     print("\nDone. All outputs saved to outputs/")

if __name__ == "__main__":
    main()
```

Listing 7.1: `scripts/run_full_pipeline.py` — End-to-end pipeline